JavaScript Core: From Basics to Advanced

JavaScript is a versatile, high-level, and interpreted programming language that is a core technology of the World Wide Web, alongside HTML and CSS. It enables interactive web pages and is an essential part of web applications.

In this guide, we will walk through practical examples of the most important concepts you need to master for development and exams.

Basic Output

The first step in any language is learning how to output data. In JavaScript, we use the `console` or the `window` object.

```
// Printing to the developer console console.log("Hello
JavaScript"); // Alerting the user alert("Welcome to
codehive!");
```

1. Variables and Data Types

Modern JavaScript uses `let` and `const` for variable declaration. `let` is for values that can change, while `const` is for constants.

```
let name = "Avni"; // String const age = 20; // Number let
isStudent = true; // Boolean // Attempting to change a const
will throw an error // age = 21; // Uncaught TypeError
```

Note: `var` is the older way to declare variables but is generally avoided in modern development due to its scope behavior.

2. Functions

Functions are the building blocks of JavaScript code. They allow you to wrap a piece of logic and reuse it throughout your program.

```
// Standard Function Declaration function add(a, b) { return a
+ b; } // Arrow Function (ES6+) const multiply = (x, y) => x *
y; console.log(add(5, 10)); // Output: 15
console.log(multiply(2, 4)); // Output: 8
```

Functions can accept parameters and return values, helping in modularizing the code structure.

3. Conditional Statements

Conditionals allow your program to make decisions based on specific criteria.

```
let age = 20; if (age >= 18) { console.log("Eligible to
Vote"); } else { console.log("Not Eligible"); } // Ternary
Operator (Shorthand) let status = (age >= 18) ? "Adult" :
"Minor";
```

This logic is essential for validating form inputs and controlling user access levels.

4. Loops

Loops are used to run a block of code repeatedly as long as a certain condition is met.

The For Loop

```
for (let i = 1; i <= 5; i++) { console.log("Iteration Number: " + i); }
```

The While Loop

```
let count = 0; while (count < 3) { console.log("Count is: " + count); count++; }
```

5. Arrays

Arrays are used to store multiple values in a single variable. They are zero-indexed collections.

```
let fruits = ["Apple", "Banana", "Mango"]; // Accessing
elements console.log(fruits[0]); // Apple // Adding an element
fruits.push("Orange"); // Array Length
console.log(fruits.length); // 4
```

Arrays come with powerful built-in methods like `map()`, `filter()`, and `forEach()`.

6. Objects

Objects are variables that can contain many values. The values are written as name:value pairs.

```
let student = { name: "Avni", age: 20, course: "CSE", greet:
function() { console.log("Hello, I am " + this.name); } }; //
Accessing properties console.log(student.course); // CSE
student.greet(); // Hello, I am Avni
```

7. Event Handling

JavaScript's interaction with HTML is handled through events that occur when the user or the browser manipulates a page.

```
// Method 1: Property Assignment const btn =
document.getElementById("myBtn"); btn.onclick = () =>
{ alert("Button was clicked!"); }; // Method 2: Event Listener
(Best Practice) btn.addEventListener("mouseover", function()
{ console.log("Mouse hovered over button"); });
```

8. DOM Manipulation

The Document Object Model (DOM) is an interface that treats an HTML document as a tree structure where each node is an object representing a part of the document.

```
// Finding an element and changing its content
document.getElementById("demo").innerHTML = "Hello World!"; //
Changing style document.getElementById("demo").style.color =
"blue"; // Creating a new element let p =
document.createElement("p"); p.textContent = "I am a new
paragraph."; document.body.appendChild(p);
```

9. Fetch API

The Fetch API provides an interface for fetching resources (including across the network). It is a more powerful and flexible replacement for XMLHttpRequest.

```
const url = "https://api.example.com/data";
fetch(url) .then(response => { if (!response.ok) throw new
Error('Network response was not ok'); return
response.json(); }) .then(data => { console.log("Success:",
data); }) .catch(error => { console.error("Error fetching
data:", error); });
```

10. Async / Await

Async/Await is a syntax that allows you to work with promises in a more comfortable fashion.

```
async function getUserData() { try { let response = await fetch('https://api.github.com/users/octocat'); let data = await response.json(); console.log(data.name); } catch (error) { console.log("Error:", error); } } getUserData();
```

It makes asynchronous code look and behave a bit more like synchronous code.

⭐ Quick Exam Note

- **Interactivity:** JS is the primary language for adding logic, events, and interactivity to web pages.
- **Integration:** It works seamlessly with HTML (structure) and CSS (presentation).
- **Core Pillars:** Mastery involves understanding Functions, Loops, Objects, and the DOM.
- **APIs:** Fetching data from external sources is a common requirement in modern apps.
- **Versatility:** JS can run on the browser (Client-side) and servers (Node.js).

Remember: JavaScript is case-sensitive and uses camelCase naming conventions for variables and functions.